

Using TLA+ as a Tool for Building Transactional Memory Models from Specialized Frameworks

Yeremey Pervunetskikh

Fac. of Computer Science

HSE University

Pokrovsky Blvd, 11, 109028, Moscow, Russia

Email: eremejperv@gmail.com

Abstract—This paper extends the Tsai-Lee framework to software transactional memory, replacing complex thread-safe architectures with modern primitives like Compare-And-Swap. By formalizing our lightweight Announce-Array protocol in TLA+, we prove the linearizability of atomic transitions under high concurrency. We demonstrate how formal specifications verify that universal algorithms maintain correctness when built from specialized components, and compare the respective specification approaches.

Index Terms—concurrent data structures, TLA+, formal verification, non-blocking algorithms, binary search trees, transactional memory.

I. INTRODUCTION

In recent decades, numerous concurrency approaches have been developed to provide high throughput with minimal system overhead. One desired example is the framework by J. Tsay and H. Lee [9], along with its adaptation for red-black trees by A. Natarajan and N. Mittal [7]. The core principle of this framework is to bring lock-free properties to binary search trees. This guarantees that at least one thread will always complete its execution in a finite number of steps. To achieve this, Natarajan and Mittal introduced a descriptor-based approach that ensures the structural consistency of the data.

In this paper, we propose a generalization of these approaches to build a Software Transactional Memory (STM) framework. Transactional memory enables non-blocking operations, which increases throughput and reduces the latency typical for traditional locking protocols. Crucially, system consistency is maintained. Furthermore, this approach introduces key advantages, such as support for nested transactions, which have no direct equivalent in lock-based systems.

The core mechanism of an STM framework relies on executing transactions, which are independent operations performed simultaneously on data. During execution, threads create local copies of the data they modify. These changes are later committed to global memory in a way that prevents interference. If a conflict occurs, the unsuccessful transactions are aborted.

Universal algorithms, derived through the transformation and generalization of specialized approaches, remain highly functional and correct. The primary objective of this work is to demonstrate how formal specifications can validate this assertion, specifically for STM and the Tsai-Lee framework.

To work with these mathematical descriptions, we propose utilizing the TLA+ specification language.

TLA+ (Temporal Logic of Actions) [11] is a formal specification language developed by Leslie Lamport for modeling concurrent and distributed systems. It enables a system to be described as a state machine, allowing its safety and liveness invariants to be verified through exhaustive state space exploration. Using TLA+ as a “tool for thinking” is a well-established practice among engineers due to its rigorous verifiability. Consequently, it was chosen to specify the generalized algorithm resulting from our transformation, thereby confirming its correctness and robustness under high contention.

II. RELATED WORKS

The protocol for software transactional memory presented here was originally described in the early work of Shavit and Touitou [4] and subsequently optimized by Guerraoui et al. [10]. Our primary focus is to formally verify this protocol and demonstrate that it can be derived from lock-free frameworks without needing generalization to the broader concept of STM. While the original specifications in the aforementioned frameworks are presented as textual descriptions with theorem-based proofs, this approach—though straightforward to implement—can introduce verification challenges. To address this, we propose using TLA+ to provide a rigorous, mathematical validation of these assertions.

Alternative STM frameworks exist, such as those by Ennals [12] and Harris [13], which rely on locking when operations conflict. In these systems, memory writes are performed by temporarily locking the target memory region, writing the values directly, and recording them in an undo log. Other strategies involve acquiring locks during the commit phase, restricting locking to that specific window. In contrast, our objective is to eliminate blocking operations entirely, allowing threads to operate independently. This independence is largely facilitated by a helping mechanism [3], a technique widely adopted across numerous non-blocking algorithms.

Furthermore, the described mechanism does not inherently support nested transactions [5]. Introducing nesting requires generalizing the concept of a “memory cell” within the global set. As detailed below, nodes can store references to operation descriptors that encapsulate their internal execution states.

However, in nested transactions, recursive helping mechanisms can cause a thread to attempt to help itself on the same set of data. To bypass this issue, one can employ references to other descriptors within the local states of existing descriptors. Although this approach is highly complex to implement, the finite nesting depth ensures it remains non-blocking and linearizable, much like the base algorithm. This holds true because double references cannot be processed alongside standard references within the same execution phase. Nevertheless, evaluating this specific extension falls outside the scope of this paper; thus, its detailed verification is not presented in this work.

III. ADVANCED TSAY-LEE FRAMEWORK

The work of Tsay and Lee [9] was developed to ensure that even complex data structures like binary search trees could support lock-free properties while remaining linearizable. Simpler structures, such as linked lists, do not require such complexity; for instance, Heller et al. [8] achieved this by utilizing logical deletion. Our goal is to guarantee that at least one thread will infinitely often complete its data structure method execution. At the same time, we require these methods to be representable as atomic operations, preserving all partial execution orders. To achieve this, a frame-based approach was proposed. In this context, a frame is defined as a subtree section that originates at a specific node and is bounded below by a set of terminal nodes. The frame size corresponds to the number of elements it contains, encompassing all edges connecting the constituent nodes of the original tree. A frame is selected such that its complete atomic replacement preserves the internal invariants of the underlying data structure.

The framework partitions the nodes accessed by each data structure operation into such frames. For example, to insert a node into the tree, the algorithm clones a frame that originates at the root node. This frame encompasses several additional nodes required to guide the insertion operation as it traverses down the data structure. Subsequently, the target operation is either performed on these nodes or they remain unchanged; the frame is then atomically replaced, provided that no other concurrent thread has modified this specific set of nodes and edges. If a concurrent modification is detected, the transformation process is retried from scratch. Otherwise, as the operation traverses deeper, a new frame is allocated. This new frame is guaranteed to overlap with the previous one, ensuring that when it is replaced by its local copy, the nodes maintain their connectivity to the root and remain continuously accessible.

This implementation was designed for the DEC Alpha architecture [14], which allowed verification of whether another thread had modified a memory element. This capability is equivalent to executing multiple atomic load-linked/store-conditional (LL/SC) instructions. Modern computer architectures, however, lack such primitives; instead, the compare-and-swap (CAS) atomic operation is predominantly used. CAS accepts three arguments: the memory location to be updated, the expected value, and the desired new value. It

possesses an infinite consensus number [3], enabling non-blocking synchronization for an arbitrary number of threads. If a thread fails to update the value due to a mismatch with the expected value, CAS returns false and no replacement occurs; otherwise, the function returns true.

Based on these operations, Natarajan et al. [1] proposed decomposing vertices and their subtrees into distinct, inter-connected nodes. Under this approach, to atomically replace a frame with its local copy, a CAS operation needs to be executed only on the single vertex from which all other vertices in that subtree section are reachable. This unique vertex serves as the root of the frame. If the CAS on the root fails, it implies that another concurrent thread has modified it first, requiring the current thread to either retry the execution or raise an error.

However, this approach introduces a critical problem previously noted by the framework's authors: if a concurrent thread modifies our frame by executing a CAS on an internal node rather than the root, data structure consistency can be compromised. Such an operation cannot detect the root-level CAS that replaced the entire frame. Consequently, the concurrent thread becomes disconnected from the main tree root because the nodes it was operating on are permanently removed from the active topology, having been replaced by our local frame copy. To prevent this inconsistency, method descriptors were introduced. To execute an operation, a thread creates a descriptor encapsulating its current data structure operation. Subsequently, a reference to this descriptor is attached to all nodes involved in the operation. If another thread attempts to access a marked node, it must first inspect the descriptor and help the original thread complete its execution. This helping mechanism must be applied recursively if executing the helping operation requires assisting a third operation, and so on. This helping mechanism, akin to the multi-word CAS approach by Ellen et al. [6], ensures both system-wide progress and strict linearizability [15] during concurrent modifications.

We formalize this frame-based update in TLA+ to ensure its correctness. The *TreeCAS* module is modeled around a minimal tree consisting of a root and its child leaf. This structure is sufficient to verify the framework's correctness, as the critical race condition to evaluate involves a larger frame overlapping a smaller one (furthermore, scaling the state space to include more nodes does not break the framework, as confirmed by our larger-scale model checking runs). To model the allocation of new nodes, we utilize an infinite memory pool constrained by a state-space filter limiting the maximum number of active allocated cells to six. The overall algorithm comprises five distinct phases, with the third phase implementing the recursive helping mechanism. This phase is structured as follows:

Cooperatively advance for descriptor if it blocks our path
 $Help(d, n) \triangleq LET otherD \triangleq nodeDesc[n] IN$
 $\wedge descState[d] = \text{"ACTIVE"}$ We can only offer help if
 $\wedge n \in descPending[d]$ we are actively trying to acquire nodes
 $\wedge nodeDesc[n] \neq NULL$ The target node must be currently
 $\wedge nodeDesc[n] \neq d$ locked by a different descriptor otherD

```

 $\wedge$  IF  $descState[otherD] \in \{\text{"ACTIVE"}, \text{"COMMITTING"}\}$ 
THEN  $CommitStep(otherD)$ 
ELSE IF  $descState[otherD] \in \{\text{"SUCCESS"}, \text{"ABORTED"}\}$ 
THEN  $ReleaseDescriptor(otherD)$ 
ELSE UNCHANGED  $vars$ 

```

As demonstrated here, we do not distinguish between the concept of an execution thread and the descriptor of the operation it performs. This abstraction can be interpreted as the use of coroutines within the physical production code, a pattern that is highly prevalent in practice. An interesting aspect of constructing formal specifications for non-blocking algorithms is that the `Help(d, n)` routine is only strictly necessary in the actual implementation. While omitting it compromises the global progress property, it is not mathematically required to maintain safety invariants.

$GlobalProgress \triangleq$

```

 $\exists d \in Descs : descState[d] = \text{"ACTIVE"} \leadsto$ 
 $\exists u \in Descs : descState[u] = \text{"SUCCESS"}$ 
 $\vee \neg FreeStructConstraint$ 

```

To verify linearizability, we enforced the preservation of the tree topology invariant, ensuring that every node remains continuously reachable from the root and is never erroneously obscured by a concurrent transaction.

IV. ADAPTATION TO TRANSACTIONAL MEMORY

A. Introducing Generalisation

In this model, locating a target node requires a top-down traversal where a thread must observe all active transactions within its path. Formally, ensuring that an update to a set is visible to all overlapping operations is non-trivial; while visibility is inherent for supersets of this set, it cannot be guaranteed for arbitrary subsets without a common synchronization point. Since no single element can consistently represent all minimal subsets (e.g., disjoint leaves), a global consensus on the frame's state is required. This formal constraint justifies reliance on localized frames and edge-based indirection as the primary mechanism for maintaining a consistent view across concurrent operations.

Consequently, we bypass the "lowest common ancestor" (LCA) approach for frame identification, as it fails to guarantee atomicity during complex rebalancing to the abstraction of frame sets. Instead, we encapsulate within the descriptor all memory cell data accessed by the current operation. This ensures that all modifications to nodes and edges are performed strictly inside this descriptor. The primary data structure positions will store exclusively references, as their values no longer remain independently valid and are not updated atomically while an operation is in progress over them.

This two-phase approach—logical commit followed by physical propagation—mirrors the Multi-word Compare-and-Swap (MCAS) semantics proposed by Harris et al. [2], ensuring that updates to multiple memory locations remain linearizable through a single shared descriptor. By its very nature, this mechanism converges toward the software transactional memory model, demonstrating that the structural primitives required to guarantee lock-free properties in search trees can

be readily transformed into a universal construction, with the only additional overhead being descriptor management. Furthermore, this design inherently preserves disjoint-access parallelism, ensuring that operations targeting non-overlapping memory regions execute independently. This property yields significant performance advantages over traditional lock-based STM implementations.

Based on this descriptor-centric architecture, we decompose each transaction into three atomic phases that ensure safe state transitions across the frame:

- **Acquire.** The thread attempts to install its descriptor D into each target memory cell $R_j \in \{r_1, \dots, r_n\}$ via $CAS(R_j, old_j, D)$. If a cell is already held by another descriptor D' , the thread must first invoke a recursive helping routine to assist the conflicting operation. This ensures that acquisition is never blocked by stalled processes. Furthermore, descriptors must be installed in an ascending order based on memory addresses to prevent deadlocks.
- **Decision.** The logical linearization point occurs here. Once all target cells are successfully acquired, the thread attempts an atomic CAS on the descriptor's status field, transitioning it from `Undecided` to either `Committed` or `Aborted`. The success of this single update determines the outcome of the entire multi-word operation.
- **Release.** In the final cleanup phase, the thread resets each node's state to a concrete value. If the status is `Committed`, the cell is updated to new_j ; otherwise, it is reverted to old_j . This phase effectively releases the descriptor-based ownership and makes the structural change visible to the rest of the system.

On the one hand, this approach closely aligns with the implementation algorithm proposed in Efficient Multi-Word Compare and Swap [10]. The proof utilized in that work relies on theorems and lemmas to demonstrate the framework's correctness. In contrast, our objective is to present a formal specification of this algorithm, optimized specifically for reader threads that require a guarantee of data immutability until the operation completes. Writing and proving such specifications mathematically is highly convenient, and their correctness can be rigorously verified using the TLA+ language and the TLC model checker.

On the other hand, this approach follows the lineage of descriptor-based atomicity and was later refined in composable memory models [5]. However, our model fundamentally alters the semantics of read operations. To guarantee a lock-free view, any process accessing a cell must proactively resolve any pending descriptor. This dependency effectively elevates a simple load into a "heavyweight" operation, as a read may now trigger a physical write to memory. While this prevents the observation of transient states, it imposes a non-trivial overhead: every read becomes a potential state transition in the global descriptor space, complicating the performance profile in read-heavy workloads.

B. Strategy For Reading Model

Shifting to a “lightweight” reading model requires a more aggressive strategy for tracking concurrent dependencies. While a similar concept has been introduced in the context of the Linux kernel’s Read-Copy Update mechanism [16], we emphasize its application here beyond search trees, extending it directly to transactional memory frameworks. We introduce an *announce array*, where each reader thread registers its presence before executing an operation. Every writing operation must inspect this array during the atomic replacement of its cell list, subsequently aborting any reader threads that are operating on vertices within that list. This validation step must occur strictly before the atomic replacement is finalized; otherwise, by the end of the check, a reader thread might complete its execution without noticing that its operation should have been aborted due to updates to the values it previously read.

Since a write operation engaged in aborting concurrent readers cannot be aborted itself, the new list of cell values intended for the atomic replacement is guaranteed to be globally valid for the entire data structure. Consequently, an incoming read operation must first register itself in the announce array before determining the current value of each target vertex. This value is either stored directly in the cell or contained within the descriptor referenced by that cell. The reader thread must retrieve this value or, if a write operation has already staged a new value, fetch the pending new value that is ready for installation. After completing its execution validly, the reader thread simply unregisters itself from the announce array to release its claim.

The safety of this optimization hinges on a fence operation that enforces a strict happens-before relationship [17] between the reader’s *Announce* step and the writer’s *Scan* phase. This ordering preserves a critical safety invariant:

- **Writer-side visibility:** If *Announce* precedes *Scan*, the writer is guaranteed to observe the reader’s entry in the announce array. Consequently, the writing thread can safely abort the conflicting reading operation.
- **Reader-side awareness:** Conversely, if *Scan* completes before *Announce*, the reader is guaranteed to observe the writer’s newly installed value (or an even newer version) during its read phase. Since the writer has already finalized its *Acquire* step, this updated view immediately triggers the corresponding validation or helping logic.

Memory barriers [18] ensure that shared variable visibility adheres to our consistency requirements, preventing hardware or compiler reorderings from introducing stale reads. This “dual-path” visibility guarantees that no race results in a lost dependency: either the writer observes the reader’s announcement, or the reader encounters the writer’s acquisition. By satisfying this property, the algorithm maintains linearizability while decoupling reads from mandatory global writes, significantly reducing contention in read-heavy workloads.

V. FORMAL METHODOLOGY

To verify the MCAS algorithm, we developed a formal specification in TLA+ suitable for the TLC model checker. The model abstracts the system into a state machine where the shared memory *mem* is represented as a record containing two fields: *mem.val*, storing the current integer value, and *mem.owner*, which holds either a NULL value or the ID of the thread currently holding an exclusive lock on the cell. To capture asynchronous execution, each thread *t* is modeled with its own control state (*pc[t]*), an announcement flag (*announce*), and a transaction status (*status*) indicating whether its operations are Undecided, Committed, or Aborted.

Need to write the part with description of specification for other fields A. Writer Lifecycle: Acquisition and Atomic Commit **Need to write the part with description of specification for the writer** B. Reader Lifecycle: Announcement and Validation **Need to write the part with description of specification for the reader** C. Formal Verification and Correctness We verify the MCAS model as a state-transition system using the TLA+ framework. The safety of the algorithm is established through the following invariants: Safety Invariant (linearisability...): System Progress (lock-free...): **Need to write the part with description of verification**

VI. EVALUATION

Need to write the part with description of specification and evaluation results

Since the execution of all data structure methods is ultimately decomposed into phases that can overlap, verification using two concurrent threads is sufficient. This setup confirms that any execution path under arbitrary instruction interleaving remains correct. Introducing a third thread merely adds a phase that overlaps with the other two in the exact same manner as they do with each other. Because execution is fully deterministic based on the descriptor at any given point, verifying the system with more than a pair of workers is unnecessary. Moreover, a larger number of threads exponentially expands the state space, making exhaustive exploration by the TLC model checker computationally prohibitive; thus, we opted to limit the configuration to two threads. Similarly, to verify whether the framework can handle overlapping frame sections correctly, a memory pool of two cells is fully sufficient.

VII. CONCLUSION

This work adapts the Tsai-Lee framework for tree-based data structures to modern universal Software Transactional Memory Model by adopting descriptor-centric architecture. Our TLA+ analysis confirms that the proposed lightweight reading protocol—utilizing an *Announce-Array* and subscription-based barriers—maintains strict linearizability while significantly reducing helping overhead. This model demonstrates a successful balance between formal robustness and modern hardware constraints. The source code for this project is available at: <https://github.com/Yerk08/MCAS-formal-specification/>

REFERENCES

- [1] A. Natarajan, L. H. Savoie, and N. Mittal, "Concurrent wait-free red black trees," in *Stabilization, Safety, and Security of Distributed Systems (SSS 2013)*, T. Higashino, Y. Katayama, T. Masuzawa, M. Potop-Butucaru, and M. Yamashita, Eds. Cham: Springer, 2013, pp. 45–60. doi: 10.1007/978-3-319-03089-0_4.
- [2] T. L. Harris, K. Fraser, and I. A. Pratt, "A practical multi-word compare-and-swap operation," in *Distributed Computing (DISC 2002)*, D. Malkhi, Ed. Berlin, Heidelberg: Springer, 2002, pp. 265–279. doi: 10.1007/3-540-36108-1_18.
- [3] M. Herlihy, "Wait-free synchronization," *ACM Transactions on Programming Languages and Systems*, vol. 13, no. 1, pp. 124–149, Jan. 1991, doi: 10.1145/114005.102808.
- [4] N. Shavit and D. Touitou, "Software transactional memory," in *Proceedings of the fourteenth annual ACM symposium on Principles of distributed computing (PODC '95)*. New York, NY, USA: Association for Computing Machinery, 1995, pp. 204–213. doi: 10.1145/224964.224987.
- [5] T. Harris, S. Marlow, S. Peyton-Jones, and M. Herlihy, "Composable memory transactions," in *Proceedings of the tenth ACM SIGPLAN symposium on Principles and practice of parallel programming (PPoPP '05)*. New York, NY, USA: Association for Computing Machinery, 2005, pp. 48–60. doi: 10.1145/1065944.1065952.
- [6] F. Ellen, P. Fatourou, E. Ruppert, and F. van Breugel, "Non-blocking binary search trees," in *Proceedings of the 29th ACM SIGACT-SIGOPS symposium on Principles of distributed computing (PODC '10)*. New York, NY, USA: Association for Computing Machinery, 2010, pp. 131–140. doi: 10.1145/1835698.1835736.
- [7] A. Natarajan and N. Mittal, "Fast concurrent lock-free binary search trees," *ACM SIGPLAN Notices*, vol. 49, no. 8, pp. 317–328, Feb. 2014, doi: 10.1145/2692916.2555256.
- [8] S. Heller, M. Herlihy, V. Luchangco, M. Moir, W. N. Scherer III, and N. Shavit, "A lazy concurrent list-based set algorithm," in *Principles of Distributed Systems (OPDIS 2005)*, M. K. Aguilera, Ed. Berlin, Heidelberg: Springer, 2005, pp. 3–16. doi: 10.1007/11795490_3.
- [9] J. J. Tsay and H. C. Li, "Lock-free concurrent tree structures for multiprocessor systems (extended abstract)," in *Proceedings of the 1994 International Conference on Parallel and Distributed Systems*, 1994, pp. 544–549.
- [10] R. Guerraoui, A. Kogan, V. J. Marathe, and I. Zlotolchi, "Efficient multi-word compare and swap," in *Proceedings of the 34th International Symposium on Distributed Computing (DISC 2020)*. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 4:1–4:18. doi: 10.4230/LIPIcs.DISC.2020.4.
- [11] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Boston, MA: Addison-Wesley, 2002.
- [12] R. Ennals, "Software transactional memory should not be lock free," Intel Research Cambridge, Tech. Rep. IRC-TR-06-052, 2006.
- [13] T. Harris and K. Fraser, "A dynamic software transactional memory system for Java," in *Proceedings of the 18th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2003)*, 2003, pp. 322–335. doi: 10.1145/949305.949334.
- [14] R. L. Sites and E. McLellan, "The Alpha AXP Architecture and 21064 Processor," *IEEE Micro*, vol. 12, no. 6, pp. 10–11, Dec. 1992. doi: 10.1109/40.180237.
- [15] M. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, 1990. doi: 10.1145/78969.78972.
- [16] P. E. McKenney and J. Slingwine, "Read-Copy Update: Using Execution History to Solve Concurrency Problems," in *Proceedings of the 11th International Parallel and Distributed Processing Symposium (IPDPS)*, 1998, pp. 509–518. doi: 10.1109/IPPS.1998.672890.
- [17] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978. doi: 10.1145/359545.359563.
- [18] P. E. McKenney, "Memory barriers: a hardware view for software hackers," Linux Technology Center, IBM Beaverton, Tech. Rep., 2010.

APPENDIX

Need to write module here, that simulates the scenario on operations of data structure, where i have the set of nodes.

Each operation creates mark on work and in each node i have either value or descriptor.

MODULE MCAS